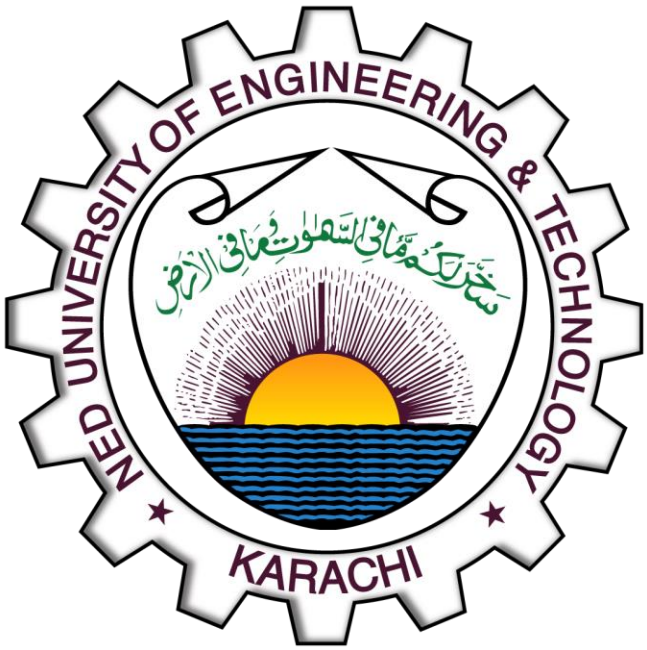
DATA STRUCTURE ALGORITHMS & APPLICATIONS

[CT-159]



NAME: TAHA AHMED MALLICK

ROLL NO: CT-25183

DEPARTMENT: BCIT **BATCH:** 2025

YEAR & SECTION: SSCS-D

LAB : #03

QUESTION#1:

CODE:

```
#include <iostream>
using namespace std;

// Implementation of Circular Queue using Circular Linked List.

class Node
{
public:
    int data;
    Node *next;

    Node(int val) : data(val), next(nullptr) {}
};

class CircularQueue
{
private:
    Node *front;
    Node *rear;

public:
    CircularQueue() : front(nullptr), rear(nullptr) {}

    void enqueue(int val)
    { // The enqueue operation in a queue always touches the rear.
        Node *newNode = new Node(val);

        if (front == nullptr && rear == nullptr)
        {
            front = rear = newNode;
            rear->next = front; // Pointing rear to front to make it
circular.
            return;
        }
        else
        {
            rear->next = newNode;
            rear = newNode;
            rear->next = front; // updating rear's next to point to front to
maintain circularity.
            return;
        }
    }
```

```
int dequeue()
{ // The elements are always removed from the front of the queue. (FIFO).
    if (front == nullptr && rear == nullptr)
        return -1;


    if (front == rear){
        int val = front->data;
        front = rear = nullptr;
        return val;
    }

    else
    {
        Node *temp = front;
        int val = temp->data;

        front = front->next;
        rear->next = front; // Updating
        delete temp;

        return val;
    }
}
bool isEmpty()
{
    return (front == nullptr && rear == nullptr);
}

void display()
{
    if (isEmpty())
    {
        cout << "Queue is empty" << endl;
        return;
    }
    else
    {
        Node *temp = front;
        cout << temp->data << " ";
        temp = temp->next;
        while (temp != front)
        {
            cout << temp->data << " ";
            temp = temp->next;
        }
        cout << endl;
```

```
            return;
        }
    }

    ~CircularQueue()
    {
        while (!isEmpty())
        {
            dequeue();
        }
    }
};

int main()
{

    CircularQueue cq;
    cq.enqueue(10);
    cq.enqueue(20);
    cq.enqueue(40);
    cq.enqueue(50);
    cq.display(); // Expected Output: 10 20 40 50.

    cq.dequeue();   // 10 is now removed.
    cq.display();   // Expected Output: 20 40 50.
    cq.dequeue();   // 20 is now removed.
    cq.enqueue(60); // 60 is added to the end of the queue.
    cq.display();   // Expected Output: 40 50 60.

    return 0;
}
```

OUTPUT:

```
10 20 40 50
20 40 50
40 50 60
```

QUESTION#2:

CODE:

```
#include <iostream>
using namespace std;

class Node
{
public:
    int data;
    Node *prev;
    Node *next;

    Node(int val) : data(val), prev(nullptr), next(nullptr) {}
};

class Deque
{
    Node *front;
    Node *rear;

public:
    Deque() : front(nullptr), rear(nullptr) {}
    void enqueueFront(int val)
    {
        Node *newNode = new Node(val);
        if (front == nullptr && rear == nullptr)
        {
            front = rear = newNode;
            return;
        }
        newNode->next = front;
        front->prev = newNode;
        front = newNode;
        return;
    }

    void enqueueRear(int val)
    {
        Node *newNode = new Node(val);
        if (front == nullptr && rear == nullptr)
        {
            front = rear = newNode;
            return;
        }
        rear->next = newNode;
```

```
        newNode->prev = rear;
        rear = newNode;
        return;
    }

    int dequeueFront()
    {
        if (front == nullptr && rear == nullptr)
        {
            cout << "Deque is empty" << endl;
            return -1;
        }

        Node *temp = front;
        int val = temp->data;
        if (front == rear)
        {
            front = rear = nullptr;
            delete temp;
            return val;
        }
        front = front->next;
        temp->next = nullptr;
        front->prev = nullptr;
        delete temp;
        return val;
    }

    int dequeueRear()
    {
        if (front == nullptr && rear == nullptr)
        {
            cout << "Deque is empty" << endl;
            return -1;
        }

        Node *temp = rear;
        int val = temp->data;
        if (front == rear)
        {
            front = rear = nullptr;
            delete temp;
            return val;
        }
        rear = rear->prev;
        temp->prev = nullptr;
        rear->next = nullptr;
        delete temp;
```

```
        return val;
    }
    bool isEmpty(){
        return (front == nullptr && rear == nullptr);
    }

    void display(){
        if(isEmpty()){
            cout << "Deque is empty" << endl;
            return;
        }
        Node* temp = front;
        while(temp != nullptr){
            cout << temp->data << "<->";
            temp = temp->next;
        }
        cout << "NULL" << endl;
    }

    ~Deque(){
        while(!isEmpty()){
            dequeueFront();
        }
    }
};

int main()
{
    Deque dq;
    dq.enqueueFront(10);
    dq.enqueueRear(20);
    dq.enqueueFront(5);
    dq.enqueueFront(15);
    dq.display(); // Expected Output: 15<->5<->10<->20<->NULL

    dq.dequeueFront();
    dq.display(); // Expected Output: 5<->10<->20<->NULL
    dq.dequeueRear();
    dq.display(); // Expected Output: 5<->10<->NULL

    return 0;
}
```

OUTPUT:

```
15<->5<->10<->20<->NULL
5<->10<->20<->NULL
5<->10<->NULL
```

QUESTION#3:

CODE:

```
#include <iostream>
using namespace std;

class Node
{
public:
    int data;
    Node *next;
    Node *prev;

    Node(int val) : data(val), next(nullptr), prev(nullptr) {}
};

class DoublyList
{

public:
    Node *head;
    Node *tail;
    DoublyList()
    {
        head = nullptr;
        tail = nullptr;
    }

    void push_front(int val)
    {
        Node *newNode = new Node(val);
        if (head == nullptr)
        {
            head = newNode;
            tail = newNode;
            return;
        }
        else
        {
            newNode->next = head;
            head->prev = newNode;
            head = newNode;
            return;
        }
    }
    void push_back(int val)
```

```
    {
        Node *newNode = new Node(val);
        if (head == nullptr)
        {
            head = newNode;
            tail = newNode;
            return;
        }
        else
        {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
            return;
        }
    }

    void insert_after(int key, int val)
    {
        if (head == nullptr)
        {
            cout << "List is empty" << endl;
            return;
        }
        Node *curr = head;
        while (curr != nullptr && curr->data != key)
        {
            curr = curr->next;
        }

        if (curr == nullptr)
        {
            cout << "Key not found " << endl;
            return;
        }
        Node *newNode = new Node(val);
        newNode->next = curr->next;
        newNode->prev = curr;
        if (curr->next != nullptr)
        { // if curr node is not at the tail node.
            curr->next->prev = newNode;
            curr->next = newNode; // Updating curr->next to now point at
newNode.
        }
        else
        { // if the curr node is at the tail node.
            tail = newNode;
        }
```

```
        return;
    }

    void pop_front()
    {
        if (head == nullptr)
        {
            cout << "List is empty" << endl;
            return;
        }

        if (head == tail)
        {
            int val = head->data;
            head = tail = nullptr;
            cout << "Deleted Value: " << val << endl;
            return;
        }

        Node *temp = head;
        head = head->next;
        head->prev = nullptr;
        int val = temp->data;
        delete temp;
        cout << "Deleted Value: " << val << endl;
    }

    void pop_back()
    {
        if (head == nullptr)
        {
            cout << "List is empty" << endl;
            return;
        }

        if (head == tail)
        {
            int val = head->data;
            head = tail = nullptr;
            cout << "Deleted Value: " << val << endl;
            return;
        }

        Node *temp = tail;
        tail = tail->prev;
        tail->next = nullptr;
        int val = temp->data;
```

```
        delete temp;
        cout << "Deleted Value: " << val << endl;
        return;
    }

    void display()
    {
        if (head == nullptr)
        {
            cout << "List is empty" << endl;
            return;
        }
        Node *curr = head;
        while (curr != nullptr)
        {
            cout << curr->data << " <-> ";
            curr = curr->next;
        }
        cout << "NULL " << endl;
        cout << endl;
        return;
    }
};

class MergeTwoList
{
public:
    DoublyList merge(DoublyList& list1, DoublyList& list2)
    {
        DoublyList mergedList;
        Node *temp1 = list1.head;
        Node *temp2 = list2.head;

        while(temp1 != nullptr){
            mergedList.push_back(temp1->data);
            temp1 = temp1->next;
        }

        while(temp2 != nullptr){
            mergedList.push_back(temp2->data);
            temp2 = temp2->next;
        }


        // Now as per question we are going to concatenate the two lists.

        return mergedList;
    }
```

```
};

int main()
{
    DoublyList list1, list2;
    list1.push_back(2);
    list1.push_back(4);
    list1.push_back(6);
    list1.push_back(8);
    list1.push_back(10);

    list2.push_back(1);
    list2.push_back(3);
    list2.push_back(5);
    list2.push_back(7);
    list2.push_back(9);

    cout << "List 1: ";
    list1.display();
    cout << "List 2: ";
    list2.display();

    cout << "Merged List: ";
    MergeTwoList mergelist;
    mergelist.merge(list1, list2).display();


    return 0;
}
```

OUTPUT:

```
List 1: 2 <-> 4 <-> 6 <-> 8 <-> 10 <-> NULL

List 2: 1 <-> 3 <-> 5 <-> 7 <-> 9 <-> NULL

Merged List: 2 <-> 4 <-> 6 <-> 8 <-> 10 <-> 1 <-> 3 <-> 5 <-> 7 <-> 9 <->
NULL
```

QUESTION#4:

CODE:

```
#include <iostream>
using namespace std;

class Node
{
public:
    int data;
    Node *next;

    Node(int val) : data(val), next(nullptr) {}
};

class Solution
{
public:
    Node *head;
    Node *tail;

    Solution()
    {
        head = nullptr;
        tail = nullptr;
    }

    void push_back(int val)
    {
        Node *newNode = new Node(val);
        if (head == nullptr)
        {
            head = newNode;
            tail = newNode;
            return;
        }
        else
        {
            tail->next = newNode;
            tail = newNode;
            return;
        }
    }

    void printll()
    {
```

```
        Node *curr = head;
        while (curr != nullptr)
        {
            cout << curr->data << " -> ";
            curr = curr->next;
        }
        cout << "NULL" << endl;
        cout << endl;
    }

    Node *getMiddle(Node *head) //function to split the list using fast &
slow pointers
    {
        if (!head)
            return head;

        Node *slow = head;
        Node *fast = head->next;

        while (fast && fast->next)
        {
            slow = slow->next;
            fast = fast->next->next;
        }
        return slow;
    }

    Node *mergeDescending(Node *left, Node *right) //function to merge two
sorted lists in descending order
    {
        if (!left)
            return right;
        if (!right)
            return left;

        Node *result = nullptr;

        // Pick the larger value to put first for descending order
        if (left->data >= right->data)
        {
            result = left;
            result->next = mergeDescending(left->next, right);
        }
        else
        {
            result = right;
            result->next = mergeDescending(left, right->next);
        }
```

```
        return result;
    }
    Node *sortDescending(Node *head) // function that implements Merge Sort
    {
        // Base case: if length is 0 or 1, list is already sorted
        if (!head || !head->next)
        {
            return head;
        }

        // Spliting the list into two halves
        Node *mid = getMiddle(head);
        Node *nextToMid = mid->next;
        mid->next = nullptr; // Break the link

        // Recursively sorting both halves
        Node *left = sortDescending(head);
        Node *right = sortDescending(nextToMid);

        // Merge the sorted halves
        return mergeDescending(left, right);
    }

    ~Solution(){
        if(head != nullptr){
            Node* curr = head;
            while(curr != nullptr){
                Node* temp = curr;
                curr = curr->next;
                delete temp;
            }
        }

}
};

int main()
{
    // Creating List N from q3 : 2->4->6->8->10->1->3->5->7->9
    Solution s;
    s.push_back(2);
    s.push_back(4);
    s.push_back(6);
    s.push_back(8);
    s.push_back(10);
    s.push_back(1);
    s.push_back(3);
    s.push_back(5);
```

```
    s.push_back(7);
    s.push_back(9);

    cout << "Original List: ";
    s.printll();

    s.head = s.sortDescending(s.head);

    cout << "Sorted List in Descending Order: ";
    s.printll();

    return 0;
}
```

OUTPUT:

```
Original List: 2 -> 4 -> 6 -> 8 -> 10 -> 1 -> 3 -> 5 -> 7 -> 9 -> NULL

Sorted List in Descending Order: 10 -> 9 -> 8 -> 7 -> 6 -> 5 -> 4 -> 3 -> 2 -
> 1 -> NULL
```

QUESTION#5:

CODE:

```
#include <iostream>
using namespace std;

class Node
{
public:
    string data;
    Node *next;
    Node *prev;

    Node(string val) : data(val), next(nullptr), prev(nullptr) {}
};

class Solution
{
public:
    Node *currpg;

    Solution(string val)
    {
        currpg = new Node(val);
    }

    void visit(string val)
    {
        Node *newNode = new Node(val);
        currpg->next = newNode;
        newNode->prev = currpg;
        currpg = newNode;
    }

    string back(int steps)
    {
        while (steps > 0 && currpg->prev != nullptr)
        {
            currpg = currpg->prev;

            steps--;
        }
        return currpg->data;
    }

    string forward(int steps)
```

```
    {
        while (steps > 0 && currpg->next != nullptr)
        {
            currpg = currpg->next;
            steps--;
        }
        return currpg->data;
    }
};

int main()
{

    Solution s("leetcode.com");

    s.visit("google.com");
    s.visit("facebook.com");
    s.visit("youtube.com");
    string url = s.back(1);
    cout << "Current URL after going back 1 step: " << url << endl; //
Expected Output: facebook.com
    cout << endl;

    url = s.back(2);
    cout << "Current URL after going back 2 steps: " << url << endl; //
Expected Output: google.com
    cout << endl;
    url = s.forward(1);
    cout << "Current URL after going forward 1 step: " << url << endl; //
Expected Output: facebook.com

    return 0;
}
```

OUTPUT:

```
Current URL after going back 1 step: facebook.com

Current URL after going back 2 steps: leetcode.com

Current URL after going forward 1 step: google.com
```